

AGENT ENGINEERING · 系统学习手册

Agent 面试 知识全景学习手册

八大板块 · 一条主线 · 全部高频考点

从「LLM 怎么调工具」到「生产级 Agent 怎么跑得稳」的完整脉络

基础概念

协议体系

记忆与状态

可靠性与安全

Harness 工程

Loop / Graph

多 Agent

可观测与成本

系统设计答法

适用 Agent / 大模型应用岗 面试冲刺 · 系统性查漏补缺

用法 先读「知识地图」建立骨架 → 逐板块精读 → 附录速查表考前自测

配套 《Agent 面试知识全景 - 思维导图》（可折叠交互版）

核心主线 LLM 只负责生成 → Agent 让它动起来 → Harness 让它跑得稳

目录

01 知识地图与学习路径

一张图看清 8 大板块的因果与递进关系，明确「先学什么、重点是什么」

02 基础概念与工作模式

LLM vs Agent · Agent vs Workflow · 四大工作模式 · 死循环防控 最高频

03 协议体系

Function Calling · MCP · A2A · Skills · 四层协作关系 最高频

04 记忆与状态系统

上下文 vs 外部记忆 · 压缩三策略 · State 三层 · Memory 两类 · 遗忘与读写

05 可靠性与安全

四大威胁 · 幻觉五类与四层防线 · 上下文漂移 · 工具调用幻觉 最高频

06 生产级工程体系

Prompt → Context → Harness → Loop → Graph · 三种 Harness 实现对比 拉开差距

07 多 Agent 架构与 Harness 治理

编排四模式 · Tool vs Agent-as-Tool · 七层治理

08 可观测性与成本优化

Agent Trace 两层设计 · 漂移检测 · 成本六招 · 混合路由

09 生产级系统设计与面试答法

五层架构 · 六个转换 · 评估四层 · 五阶段落地 · 答题框架

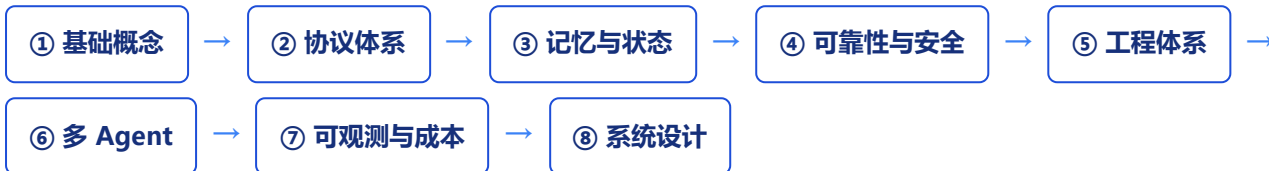
010 附录：高频面试题速查 · 记忆口诀 · 关键数字 · 术语表

考前 30 分钟的最佳材料

01 知识地图与学习路径

本手册的全部内容，可以用一句话串起来：**LLM 只能生成文本，Agent 通过「工具 + 记忆 + 规划 + 循环」让它动起来，而 Harness 工程让它从「能跑通一次」变成「能稳定跑很多次」**。八大板块不是并列的知识点堆，而是层层递进的因果链。

1.1 总览：八大板块的递进关系



读法：① 解决「Agent 是什么」；② 解决「Agent 怎么接到外部能力」；③ 解决「Agent 怎么记住」；④ 解决「Agent 怎么不出事」；⑤ 解决「循环本身怎么设计」；⑥ 解决「多个 Agent 怎么协作」；⑦ 解决「线上怎么看清、怎么省钱」；⑧ 把前面全部收进一套可交付的架构与答法。

板块	回答的核心问题	一句话抓住	面试价值
① 基础概念	Agent 是什么？和 LLM、Workflow 差在哪？	控制权归谁：代码（Workflow）还是 LLM（Agent）	开场必问，答不好直接掉档
② 协议体系	Agent 靠什么调用外部世界？	FC 是机制，MCP 是标准，A2A 是横向协作，Skills 是领域知识	区分「看过文章」和「真懂」的分水岭
③ 记忆与状态	Agent 怎么不失忆、不串味？	State 管当前任务，Memory 管跨任务经验，两者必须分开	中高级岗必问
④ 可靠性与安全	Agent 胡说、越权、死循环怎么办？	幻觉四层防线：Prompt → 工具 → 证据 → 输出校验	高频追问，能讲透就赢一半
⑤ 工程体系	怎么让长链路稳定跑？	Prompt ⊂ Context ⊂ Harness；Loop 是 Harness 的心脏	拉开与普通候选人的差距
⑥ 多 Agent	什么时候该拆多个 Agent？	Agent 负责局部智能，Harness 负责全局控制	架构岗重点
⑦ 可观测与成本	线上不稳、账单爆炸怎么治？	没有 Trace 就没有优化；80% 的任务用不着大模型	落地能力证明
⑧ 系统设计	你能否设计一个生产级 Agent？	目标 → 架构 → Harness → 治理 → 组织	终面压轴

1.2 三条贯穿全书的主线（考前必记）

主线一：从「说」到「做」

LLM 只会生成 token → Function Calling 让它输出调用指令 → MCP 把接入标准化 → Agent 循环把一次调用变成多步任务。这条线回答「Agent почему能干活」。

主线二：从「能跑」到「跑得稳」

Prompt → Context → Harness → Loop → Graph。控制的粒度越来越粗：从一句话，到一个循环，再到一张显式控制图。这条线回答「生产级差在哪」。

主线三：从「单点」到「系统」

单 Agent → 多 Agent → 平台化。每一步都要求：状态外化、权限治理、评估与观测。这条线回答「架构怎么演进」。

学习路径建议（按面试性价比排序）

第 1 遍（2 小时）：读第 2、3 章 —— 基础概念 + 协议体系，这两块是 100% 会问的地基。

第 2 遍（3 小时）：读第 5、6、7 章 —— 可靠性、工程体系、系统设计，这是拉开档次的关键。

第 3 遍（1 小时）：读第 4、8 章 —— 记忆状态、可观测成本，补齐细节。

考前 30 分钟：只看第 10 章附录速查表 + 每章的「核心考点」绿框。

全书最强记忆锚点

$\text{Agent} = \text{Model} + \text{Harness}$ ($\text{Harness} = \text{Agent} - \text{Model}$)

所有工程话题都可以归结为一句话：**模型不变，把 Harness 做厚。**

02 基础概念与工作模式

最高频

本章回答面试的第一组问题：**Agent 到底是什么？它和 LLM、和 Workflow 有什么区别？它是怎么循环的？**这部分答不清，后面再深也没用。

2.1 LLM vs Agent

LLM 的本质

LLM 是一个**条件概率模型**： $P(\text{token}_n \mid \text{token}_{1..n-1})$ ，本质是**无状态的函数**——给一段输入，输出下一个 token 的分布，仅此而已。它不记得你上一次问过什么，也不能改变任何外部状态。

LLM 的四大天花板（考点：为什么需要 Agent）

天花板	具体表现	Agent 的补法
只会说不会做	不能操作外部世界，不能查库、不能发邮件、不能改文件	工具模块（Tool / Function Calling）
没有记忆	上下文窗口一满就“失忆”，跨会话完全不认识你	记忆模块（短期上下文 + 长期外部记忆）
知识截止	不知道昨天发生的事，也无法访问内部私有数据	检索 / 工具调用实时取数
不会规划	只会线性作答，无法把复杂目标拆成多步并调整	规划模块 + 执行循环

Agent 的构成：LLM + 工具 + 记忆 + 规划

LLM（大脑）

理解意图、推理判断、生成决策。它是唯一的“思考者”，但本身不执行任何动作。

规划模块

任务拆解、步骤排序、必要时重新规划。把“一个目标”变成“一串可执行步骤”。

记忆模块

短期：当前上下文；长期：向量库 / 关系库 / KV。解决跨步、跨会话的信息保持。

工具模块（手和脚）

外部 API、数据库、代码执行器、文件系统。Agent 与真实世界唯一的接口。

核心考点：一句话讲清区别

LLM 告诉你怎么做，Agent 直接帮你做完。

更技术化的表述：LLM 输出的是「文本」，Agent 输出的是「状态变化」。Agent 在 LLM 之上增加了**工具（能动作）、记忆（能累积）、规划（能拆解）和循环（能迭代）**四个能力。

2.2 Agent vs Workflow

这是最容易被追问、也最容易答偏的一组对比。关键只有一句：**控制权在代码，还是在 LLM。**

维度	Workflow（工作流）	Agent（智能体）
控制权	流程写死在代码里，每一步由代码决定	LLM 接收目标后自主规划执行路径
LLM 的角色	只是某个节点的“处理器”	是决策者，决定下一步做什么
Token 消耗	约 1x	约 4-8x
可预测性	高 —— 路径固定	低 —— 每次可能不同
灵活性	低 —— 只能处理预设分支	高 —— 能应对开放式目标
调试难度	易 —— 有明确执行路径	难 —— 需要 Trace 才能还原
适合任务	固定流程、强一致性、延迟敏感	开放式目标、需要判断与调整

必答的加分观点

生产里**混合架构最常见**：用 Workflow 提供稳定骨架（确定性的流程、校验、权限节点），把 Agent 放在需要判断和应对异常的环节。

结论话术：“不是二选一，而是**外层确定性 Workflow + 内层有限自治 Agent。**”

2.3 四大工作模式 **必背**

① ReAct (Reasoning + Acting)

最基础、最主流的范式，本质是一场 **Thought → Action → Observation** 的循环，直到模型认为可以给出答案。



优点	说明
透明可审计	每一步思考与动作都在文本里，人能看懂它在干嘛
灵活适应	根据观察结果动态调整下一步，不依赖预设分支
通用性强	几乎不需要为任务定制流程，LangChain / LangGraph 默认选择

缺点：Token 消耗大（每步都要把历史重新喂一遍）、可能陷入死循环、多轮往返导致延迟高。

最常见的误区

ReAct 不是一个框架，也不是模型能力 —— 它本质是一种 Prompt 范式，循环的对象自始至终是「一段不断变长的文本」。理解这一点，才能理解第 6 章为什么说“ReAct 在生产里撑不住”。

② Plan-and-Execute (先规划再执行)

两阶段：Planner 一次性产出完整计划 → Executor 逐步执行，把「规划」和「执行」彻底解耦。

对比项	ReAct	Plan-and-Execute
相对 Token 消耗	100%	约 20%
执行粒度	边想边做，每步都调 LLM	计划一次生成，执行阶段少调用
主要风险	死循环、Token 爆炸	计划不准，一条错步步错

标准解法

在执行过程中加「重新规划检查点」：执行 N 步或遇到与预期不符的观察时，回到 Planner 重新出计划。

③ Reflection (自我反思)

两个角色来回迭代：Writer 生成 → Reviewer 审查 → 根据意见修改，直到质量达标。

- 适用场景：代码生成、法律文书、学术论文、创意写作 —— 共同点是「质量可评判、允许多轮打磨」。
- 进阶用法：可用于“验证反思”，让 Reviewer 去核对事实，从而校正幻觉。
- 代价：Token 与延迟都会成倍增加，所以只在「质量优先、成本不敏感」的场景使用。

④ Multi-Agent (多智能体协作)

结构：Orchestrator 协调 Research / Coder / Reviewer 等专职 Agent 并行工作。

优点	缺点
专业化：每个 Agent 有自己的系统提示词与工具权限	协调复杂度高：任务分解、上下文传递、结果冲突都要额外设计
并行化：独立子任务可同时跑，缩短总时长	成本高：每次子 Agent 都是一次独立模型调用
上下文隔离：互不干扰，避免污染	调试难：跨 Agent 的日志追踪难一个量级

必须说出的话

不要过早引入 Multi-Agent。单 Agent + 好工具能解决绝大多数问题；只有当出现「上下文互扰、需要独立规划、需要并行加速、需要专业分工」时才拆。多 Agent 的复杂度是乘性的，不是加性的。

2.4 死循环防控 (高频追问)

ReAct 循环最大的工程风险就是转不出来。三种最有效的防线：

手段	典型阈值	说明
最大步数限制	通常 15 步 ；Loop 类场景 15-50	超过硬上限直接终止并返回当前最好结果
重复动作检测	连续 3 次 相同工具 + 相同参数	判定为原地打转，立即退出或强制换策略
超时控制	整个任务级最大执行时间	与步数限制互补，防止单步卡死拖垮整体

补充加分

还可以加：预算上限（Token / 成本）、终止信号结构化校验（要求模型输出明确的 done 字段）、外部校验器判定任务是否真的完成。

2.5 Agent 自身的演进（递进脉络）



本章核心考点

① Agent 与 LLM 的本质区别是「有无工具/记忆/规划/循环」；② Agent 与 Workflow 的分界是「控制权归谁」；③ 四大模式要能各自说出适用场景和代价；④ 死循环三件套：步数、重复检测、超时。

03 协议体系

最高频

本章四层协议是面试最容易「一问就露馅」的地方。记住它们的**分工**，比记住定义重要得多：**FC 是机制**，**MCP 是标准**，**A2A 是横向协作**，**Skills 是领域知识**。

3.1 Function Calling（工具调用的底层机制）

本质：LLM 输出一段**结构化的 JSON 调用指令**，模型自己不执行任何东西。

关键认知（一定要说这句）

模型只决定「**调哪个工具、传什么参数**」，真正的执行发生在**应用侧代码**里。模型输出的是「调用意图」，不是「调用结果」。

四步流程

#	步骤	发生了什么
1	定义工具	在请求里带上 <code>tools</code> ：name / description / parameters (JSON Schema)
2	LLM 生成 <code>tool_calls</code>	模型判断需要调用，输出结构化 JSON（工具名 + 参数）
3	应用执行	你的代码解析 <code>tool_calls</code> ，真正去调 API / 查库 / 跑代码
4	结果回传	以 <code>role: tool</code> （或 <code>tool_result</code> ）把结果喂回，模型生成最终回答

并行调用 Parallel Function Call

一次响应里返回多个 `tool_calls`，应用可以并发执行：

串行 $T1 + T2 + T3$ → 并行 $\max(T1, T2, T3)$

延迟优化的关键手段，前提是多个调用之间没有数据依赖

厂商格式差异（细节考点）

厂商	工具定义	调用返回	结果回传
OpenAI	<code>tools</code> 数组	<code>tool_calls</code> 数组	<code>role: "tool"</code>
Claude	<code>input_schema</code>	<code>tool_use</code> block	<code>tool_result</code> block

为什么说 FC 是整套体系的基石

它解决了两个最根本的问题：**何时调**（模型自己判断）与**传什么参数**（模型按 Schema 生成）。
更关键的是：**Agent 的 ReAct 循环，本质就是 Function Calling 的循环编排**——Thought 是模型输出，Action 是 tool_calls，Observation 是 tool 结果。

3.2 MCP (Model Context Protocol)

解决的根本问题：工具集成的**N×M 爆炸**。N 个 AI 应用 × M 个工具服务，若两两对接需要 N×M 个适配器；有了统一协议后，只需 **N + M**。

三个角色

MCP Host

AI 应用本身：Claude Desktop、Cursor、自研 Agent。它是用户直接交互的对象。

MCP Client

住在 Host 内部，负责与 MCP Server 建立连接、收发消息。一个 Host 可以有多个 Client。

MCP Server

对外暴露工具能力的服务端，把某个系统（数据库/GitHub/文件）包装成标准能力。

三类资源（并列）

类型	是什么	例子
Tools	可执行操作（会改变状态）	发消息、查数据、写文件
Resources	可读数据源（只读）	文档、代码库、数据库内容
Prompts	预设提示词模板	代码审查模板、报告生成模板

工具发现机制（考点）

启动时 Host 向每个 Server 发 `tools/list`，把返回的工具清单注入 LLM 上下文。这意味着 Agent 可以**运行时动态发现新能力，无需重新部署**。这是 MCP 相比硬编码工具的最大价值之一。

三层安全（递进）



底层协议与生态

- **传输**：本地用 `stdio`，远程用 `HTTP + SSE`。
- **消息格式**：`JSON-RPC 2.0`。
- **生态现状**：已成事实标准，生态最丰富。

容易答错的点

MCP 只解决工具接入，不解决 Agent 之间的协作，也不解决权限治理。标准化 ≠ 安全 —— 它提供能力，治理必须由 Harness 承担（见第 6 章第 6 层）。

3.3 A2A (Agent-to-Agent)

解决：Agent ↔ Agent 的横向协作。为什么 MCP 解决不了？因为 MCP 的设计目标就是「工具」，不是「Agent」—— 工具是被动执行体，Agent 是有自己规划能力的协作方。

核心概念	是什么	作用
Agent Card	智能体名片	声明名称 / 能力 / 端点 / 认证方式，供对方发现与调用
Task	标准任务对象	状态机：CREATED → PROCESSING → COMPLETED / FAILED
Message & Artifact	过程沟通 / 最终成果	Message 用于多轮沟通，Artifact 是交付物

MCP vs A2A (标准答法)

MCP 解决纵向：Agent ↔ 工具；A2A 解决横向：Agent ↔ Agent。两者互补而非替代，一个 Agent 完全可以同时是 MCP Client 和 A2A Server。

技术基础：HTTP + JSON + SSE，无需新基础设施。生态现状：Google + 50 多伙伴，处于早期快速发展阶段。

3.4 Skills (能力模块)

定义：跨任务复用的可执行能力模块。两个关键词：可复用、能力模块。它不是“更长的 Prompt”。

对比	左边是什么	Skill 是什么
vs Prompt	解决单次任务的表达	解决跨任务的复用
vs Few-shot	教格式（照着这个长这样）	教方法论（这类问题该怎么想、怎么做）

九大构成要素

① 适用场景 ② 不适用场景 ③ 输入要求	④ 操作步骤 ⑤ 工具使用原则 ⑥ 输出格式	⑦ 质量标准 ⑧ 失败处理 ⑨ 安全边界
-----------------------------	------------------------------	----------------------------

文件结构：身份定位 + 工作流程 + 注意事项 + 输出规范。

必须点明的边界

Skill 是软约束 —— 它写在自然语言里，模型可能不遵守。**权限与审批必须在工程层强制执行**（IAM、Schema、审批流），不能指望 Skill 文件管住高风险动作。

3.5 四层协议的协作关系（递进）

层次	解决什么	一句话
Function Call	调用工具的基础机制	怎么调
MCP	工具接入的标准化	用什么（工具从哪来）
A2A	Agent 间协作	和谁协作
Skills	领域知识编码	怎么想（方法论）

Skills 决定「怎么想」 → MCP 决定「用什么」 → FC 决定「怎么调」

A2A 负责把这些能力在多个 Agent 之间横向连起来

04 记忆与状态系统

Agent 的两个“失忆症”：**上下文一满就忘**、**跨会话不认识你**。本章解决的就是“怎么记、记多少、什么时候忘、什么时候读”。

4.1 两个层次：上下文记忆 vs 外部记忆

层次	特点	存什么
上下文窗口 In-Context	速度最快；容量有限（通常约 128K tokens ）；任务结束即消失	当前对话、任务状态、加载的 Skill、工具调用历史
外部记忆 External	容量近乎无限；需要一次检索往返；可跨会话持久	见下方三种存储分工

存储类型	存什么	检索特性
向量数据库	用户偏好、历史经验、文档切片	语义检索、模糊匹配（“意思相近”也能找到）
关系数据库	结构化事实、用户档案	精确查询（按字段、按条件）
KV 存储 (Redis)	任务状态、中间结果	极快读写，适合高频短生命周期数据

4.2 记忆压缩三策略（并列）

策略	做法	取舍
滑动窗口	丢弃最旧消息，只保留最近 N 条	简单、快；但会丢掉早期关键信息
摘要压缩	用 LLM 把旧对话总结成一段	Token 大幅缩减；有一次额外模型调用成本
重要性过滤	只保留关键信息，丢弃过程细节	信息密度高；判断“什么是关键”本身较难

配合使用

压缩后的内容应当**归档到外部记忆**，需要时再检索回来——即“压缩不是删除，而是降级存储”。

4.3 State vs Memory（必须分清）

	State (状态)	Memory (记忆)
是什么	当前任务的运行数据	跨任务复用的经验
生命周期	短（任务结束即清理）	长（持续累积）
核心关心	一致性（不能前后矛盾）	相关性（检索出来的经验要管用）

State 三层（递进）

层级	内容	存放与生命周期
Working State	当前步骤的临时上下文	内存中，任务结束即丢弃
Session State	一次会话内多 Agent 共享的数据	Redis + TTL，会话结束过期
Execution Log	不可变的执行日志	持久化，用于审计 / 回放 / 评估

Memory 两类（并列）

Episodic Memory 事件记忆

“踩过的坑、用户偏好、上次怎么处理的”。

回答：过去发生过什么。

Semantic Memory 语义记忆

领域概念、业务规则、工具约束。

回答：这个世界是怎么运作的。

4.4 注入时机

方式	做法	问题
全量前置注入	任务开始就把相关记忆全部塞进上下文	简单稳定，但费 Token
按需检索	给 Agent 一个 <code>memory_search</code> 工具，需要时才查	省钱，但 Agent 可能忘记调用
混合（推荐）	前置注入少量高置信记忆 + 按需检索补充	兼顾成本与可靠性，工程最常用

4.5 遗忘机制与读写时机

保留分数（决定一条记忆的存亡）：

保留分数 = 访问频次 + 创建时间 + 重要性 + 最近使用 + 是否仍有效

分数区间	处理方式
低分	直接删除
中分	压缩成摘要保留
高分	保留原文

写入时机

任务完成存结果 · 用户给信息存偏好 · 发现新知识更新 · 出错存失败原因

读取时机

任务开始加载偏好 · 遇陌生问题检索经验 · 事实核查检索已知

本章核心考点

① State 与 Memory 的区别（一致性 vs 相关性）；② 压缩三策略；③ 混合注入是工程最优；④ 记忆必须会“遗忘”，否则噪声会淹没信号。

05 可靠性与安全 最高频

Agent 的四大事故类型：**被注入、越权、泄密、烧钱**。本章给出威胁清单、防御手段、幻觉治理的四层防线，以及上下文漂移的检测与解法。

5.1 四大安全威胁（并列）

威胁	场景	后果
Prompt Injection	网页 / 文档内容里藏了恶意指令，模型照做了	被诱导执行未授权操作
权限越界	原本只该读数据，被诱导去删数据	数据被破坏
数据泄露	通过工具调用把敏感信息发给第三方	合规事故
资源滥用	陷入死循环，疯狂调用 API	费用爆炸

5.2 核心防御

① 最小权限原则

- 读任务只给 `SELECT`，绝不给 `DELETE`。
- 生产 / 测试使用不同凭证，避免误操作打穿生产。
- 权限**跟着动作走**，**不跟着 Agent 名字走**。

② Human in the Loop（人在回路）

高风险操作前暂停，请求人类确认。**必须有审批清单：**

必须审批的动作	原因
删除数据	不可逆
发送外部邮件 / 消息	对外不可撤回
修改权限配置	可能扩大攻击面
超过阈值的资金操作	直接经济损失

③ Prompt Injection 防御四手段

数据 / 指令分离

外部内容只能作为“数据”，不能作为“指令”被解释

输入过滤

检测“忽略之前指令”“你现在是……”类可疑内容

Prompt 模板隔离

用户输入不直接拼进系统提示词，改用模板占位

上下文标记

明确标注哪段是外部数据、哪段是系统指令

5.3 可靠性设计四件套（并列）

手段	定义	解决什么
幂等性	同一操作执行多次结果相同	重试导致的重复提交（如重复下单）
回滚机制	重要操作前先备份，支持撤销	错误动作无法挽回
超时控制	每个工具调用设超时，整体任务设最大时间	卡死拖垮整个任务
降级策略	工具失败有备用方案，不盲目重试	失败后被无限重试放大成本

5.4 幻觉治理：五类幻觉与四层防线 高分项

Agent 幻觉的五种类型

类型	表现	危险度
知识幻觉	编造不存在的事实	中
工具幻觉	调用不存在的工具	中（可拦截）
参数幻觉	参数不符合 schema	中（可校验）
证据幻觉	无来源却说“根据文档可知”	高（难被发现）
行动幻觉	该审批没审批、只读变写入	最危险

四层防线（递进，必须按顺序讲）



第 1 层 • Prompt 约束

能做：声明规则 —— 不知道就说不知道、只基于证据回答、高风险操作先确认。

边界（关键）：自然语言约束只是「希望遵守」，不是强制执行。所以它只是第一层，绝不能是唯一一层。

第 2 层 · 工具约束

- **工具白名单**：只暴露当前任务真正需要的工具，减少选错空间。
- **描述写清能力边界**：明确标注“只读”“不能做什么”。
- **参数必须有 JSON Schema**：类型、枚举、必填、格式，让错误在调用前被拦。
- **Observation 必须来自真实工具结果**，不允许模型自编（直接掐断工具幻觉）。
- **高风险工具**：二次确认 + 权限校验 + 阈值 + 审批 + 幂等回滚，五件套齐上。

第 3 层 · 证据约束

- **先检索再回答**，不让模型凭记忆答业务事实。
- **强制引用来源**，把结论和证据绑定（可追溯才能被验证）。
- **检索不到就拒答**——“当前资料中未提及”，拒答比编造好一万倍。
- **Rerank 降低噪声**，只把最相关的证据给模型。
- **答案与证据一致性校验**：规则 or LLM-as-Judge。

第 4 层 · 输出校验

- **结构化输出校验**：字段 / 枚举 / 置信度阈值。
- **业务规则校验**：用确定性代码兜底，不让概率模型拍脑袋。
- **敏感内容校验**：隐私、越权建议、把建议说成事实。
- **双阶段审查**：Generator + Reviewer（仅高风险场景才上，因为贵）。

追问：「约束都加了还是幻觉怎么办？」

先拦截：无证据 / 低置信 / 校验失败 / 参数非法 / 高风险未确认 → 一律拦下。

再降级：换强模型 → 只返回原始证据 → 让用户补充 → 转人工 → 写操作直接停止。

高风险动作：审批流 + 幂等 + 回滚 + 操作日志 + 权限隔离，全部到位。

必须记 Trace：Prompt / 检索文档 / 工具参数与返回 / 最终输出 / 失败原因。

长期治理四件事（组织视角，加分项）

- ① **归因分类**：定位到 Prompt / 工具描述 / Schema / 召回 / Rerank / 输出校验；
- ② **事故样本进 Eval**，每次改动跑回归；
- ③ **建监控指标**：无证据回答率、工具失败率、参数校验失败率、人工接管率；
- ④ **分场景设安全等级**：闲聊轻、业务咨询要证据、写操作要审批。

5.5 上下文漂移

根因（从注意力机制理解）

- **近因效应**：模型倾向于给最近的 Token 更高权重，越早的信息越被忽略。
- **中间结果抢焦点**：上下文越长，干扰信号越多，原始目标被淹没。
- **本质**：原始目标在注意力分配中**逐渐失焦**。

三种漂移模式（并列）

模式	表现	特点
目标漂移	从任务 A 滑到任务 B	最容易察觉
优先级漂移	支线挤占主线，主次倒置	中等
风格漂移	输出格式悄悄改变	最隐蔽

五个检测信号

① 当前动作与原始目标关联度下降

② 步骤重复率升高

③ 目标完成进度长期为 0

④ 目标表述发生变化

⑤ 关键证据未被使用

解法三层（递进）+ 终极解药



Context Reset（终极解药）

直接把旧上下文窗口整个丢掉，换一个干净窗口接手任务。

前提：状态必须全部外化到文件系统，否则一重启就全丢。

原则：**重启胜过修补，状态沉到文件里。**

5.6 工具调用幻觉

根因（一句话）：模型选工具**不是查表，而是在概率预测“下一个最可能的工具名”**。这个理解是解题关键。

类型	根因	解法
调用不存在的工具	工具描述与任务描述之间存在语义缝隙	工具描述结构化 + Few-shot + 注册表校验工具名
参数类型 / 格式错误	类型与格式约束未声明，模型按自然语言习惯填	JSON Schema + 结构化输出 + 调用前类型检查
无意义的工具调用	模型有“工具使用倾向”，被训练得过于积极	加“无需调用”选项 + 调用必要性判断 + 频率阈值

通用防线：三段式校验（递进）

阶段	做什么
调用前	校验工具名 / 参数 Schema / 调用必要性
调用中	超时与异常捕获，错误结构化后再回传模型
调用后	校验返回格式，重试 2-3 次后降级处理

工具调用故障排查五步（必背顺序）

工具是否存在 → 权限是否允许 → 参数是否通过 Schema → 工具服务是否成功 → 模型是否正确理解返回结果

顺序本身就是答案：从最确定、最便宜的地方开始排除。

06 生产级工程体系 拉开差距

这是整套知识里最能体现深度的一章。**三次重心转移 + 四个 Engineering 概念**，把「让模型听懂」推进到「让循环跑得稳」。

6.1 三次重心转移（递进）

阶段	解决什么	一句话
Prompt Engineering	“怎么说”	让模型听懂你想干啥
Context Engineering	“给什么信息”	让模型知道该用什么
Harness Engineering	“能不能持续做对”	把环境做对，让错误永不再犯
Learning Loop（第四层）	“越干越强”	让系统从每次执行中自我改进

核心认知：包含而非替代

Prompt $\subset$ Context $\subset$ Harness —— 后者包含前者，不是取代前者。

分水岭判断：单轮对话靠 Prompt；需要外部知识靠 Context；**长链路、低容错靠 Harness**。

6.2 Harness Engineering

概念来源与核心定义

- 来源：Mitchell Hashimoto 《My AI Adoption Journey》第 5 步 “Engineer the Harness”。
- 核心定义（务必逐字理解）：**Agent 每犯一次错，就把修复沉淀到环境里，让它永不再犯。**
- 官方背书：OpenAI 官方博客《Harness engineering: leveraging Codex》。

$$\text{Agent} = \text{Model} + \text{Harness}$$

即 Harness = Agent - Model：除了模型本身，其余一切都是 Harness

六层核心组件（并列）

侧	组件	要点
输入侧	上下文精细化	钉死角色与目标、动态筛选 (just-in-time retrieval) 、结构化组织
	记忆与状态	状态外化到文件系统，分层存 (任务状态 / 会话中间结果 / 长期记忆)
动作侧	工具系统	只给真正需要的、该查才查、结果先提炼再喂回
	执行编排	循环工程化，给模型明确的工作轨道
校验侧	评估与观测	Eval 集 + Trace，把调试从 “猜” 变成 “看”
	约束与恢复	约束硬编码、每步校验、失败有预案

看得准（输入） → 做得对（动作） → 错了能兜底（校验）

五个真实难题与工程原则（面试最爱追问）

难题	根因	工程解法
跑久了越走越偏	上下文焦虑	Context Reset —— 重启胜过修补
自评总是偏乐观	自己评自己没有约束	三角分工 ：Planner / Generator / Evaluator，生产验收必须分离
反复失败怎么破	环境不提供反馈信号	别催模型，补环境 (lint / 单测 / 运行环境)
规范文件越长越糊涂	信息过载	渐进式披露 ，AGENTS.md 约 100 行 的目录页
技术债越堆越烂 (AI slop)	只加功能不还债	Golden Principles + 后台 Agent 天天自动还

与 MLOps 的区别

MLOps 管 “怎么把模型搞上线” ； Harness 管 “上线后怎么跑得稳” 。两者是接力关系，不是同一件事。

6.3 Loop Engineering

起源：ReAct 是 2022 年的 Prompt 范式，在生产环境撑不住。

ReAct 的五个裂缝（并列）

裂缝	后果
上下文越滚越长	Token 涨 + 注意力稀释
状态全靠上下文记	截断即丢，是任务漂移的源头
死循环	模型“看不见”自己在重复
终止判断不可靠	模型说停就停（偷懒）或停不下来
不可观测	出问题只能人肉从文本里扒

核心定义

Loop Engineering = 把循环本身当作需要设计、控制、观测的工程系统。

Agent 到底在循环什么：五个变量（必背）

变量	工程要点
上下文	要选、要压、要截（超阈值 Auto-Compact）
状态与记忆	独立存、跨圈传递、单独钉住原始目标
预算	步数 / Token 成本 / 时间，多重上限，触顶优雅退出
可用工具集	按阶段动态收放，危险工具加权限门
终止与收敛	结构化完成信号 + 外部校验 + 硬兜底，三层叠加

ReAct vs Loop（对比）

维度	ReAct	Loop Engineering
循环对象	一段不断变长的文本	上下文 + 状态 + 预算 + 工具 + 终止的系统状态
工程重心	调 Prompt	设计循环每个环节怎么转、怎么收敛
预算概念	没有	多重上限
可观测	一坨文本靠人肉扒	每圈结构化记录，可回放

与其他 Engineering 的关系（嵌套）

概念	职责
Context Engineering	管往循环里喂什么
Loop Engineering	管循环这个控制结构本身怎么转、怎么收敛
Harness Engineering	把循环 + 上下文 + 工具 + 记忆 + 安全 + 可观测全包住的外壳

关系一句话

它们是**嵌套关系**：Harness 是外壳，Loop 是 Harness 的心脏。

6.4 Graph Engineering

定义：把 Agent 系统设计成一张**显式的控制图**，而不是把决策都塞进一个隐式的 while 循环。

三主角（并列）

节点 Node

能独立完成、独立测试的能力单元。越“无聊”越好——职责单一才可测。

边 Edge

下一步的规则：确定边 / 条件边 / 模型边。

状态 State

随边传递的结构化数据 + 检查点 Checkpoint。

为什么 Loop 会长成图

因为出现这些需求：**并行、分工、校验、人工确认、断点恢复**。

关键表述（面试可直接用）：「Loop 没死，它被降到节点内部了。」

生产里真正值钱的五个能力（并列）

能力	说明
并行与汇合	fan-out 拆分 + fan-in 汇总
失败隔离与恢复	坏一个节点只重试一个节点（前提：幂等与可重试）
人在回路	人作为节点：有输入边、等待、批准 / 拒绝两条输出边
预算与安全	闸门放在边上，阻止边被跨越
轨迹评估	看经过哪些节点、为何选这条路、耗时成本、是否走过关键校验

概念区分（极易混淆，务必分清）

概念	管什么
Graph Engineering	控制图——管任务下一步怎么走
GraphRAG	知识图谱——管知识怎么找、怎么关联
workflow编排	确定业务流程（ETL / 订单 / CI-CD）
Harness Engineering	给 Agent 提供完整的运行外壳

何时该用图

有真并行 / 有明确分支 / 有独立校验 / 有长时间等待 / 有高风险操作 / 有高失败成本

何时不该用

路径短且开放的问答类任务，先用一个治理好的 Loop 就够了。
图多一条边，就多一条要测试、监控、恢复的路径。

6.5 三种 Harness 实现对比（横向视野加分项）

维度	OpenClaw（小龙虾）	Hermes Agent（爱马仕）	Claude Code
定位	全平台个人 AI 助手，消息优先、本地优先	自进化 Agent，“The agent that grows with you”	产品级编程 Agent，工程化极致、安全最完善
技术栈	TypeScript；一个网关打通 25+ 消息平台	Python；核心差异化是学习闭环	<code>while</code> 循环 + Explore / Plan / General-purpose 三种子 Agent
上下文	AGENTS.md / SOUL.md / TOOLS.md 全量注入	just-in-time retrieval + 分层注入（始终 / 按需 / 动态替换）	200K 窗口 + 三层压缩 + Context Reset
记忆	本地文件持久化， 无跨会话记忆 （每次都是“新手”）	最深：持久化 + Honcho 用户建模 + FTS5 跨会话搜索 + 自我督促	CLAUDE.md 分层注入 + <code>.claude/</code> 状态外化 + <code>~/.claude/</code> 长期记忆
工具	MCP + ClawHub 社区技能市场；沙箱 Docker / SSH	MCP + 自生成技能 + 动态白名单；6 种终端后端 + cron	MCP + Hook 机制；23 层顺序检查
安全	沙箱隔离，粒度粗（全有或全无）	动态白名单 + 自生成技能需审查	<code>deny > ask > allow</code> 优先级

三个必须记住的细节

- ① **Hermes 的学习闭环**：执行任务 → 总结经验 → 生成 skill → 存入记忆 → 下次复用。
- ② **Claude Code 的设计巧思**：CLAUDE.md 作为「用户消息」注入而非 System Prompt，**防止把安全规则覆盖掉**。
- ③ **Hook 四事件**：PreToolUse / PostToolUse / Notification / Stop。

共性	演进主线
都是 Harness Engineering 的具体实现，都基于 ReAct 循环 + MCP 工具	能干活（OpenClaw）→ 越干越强（Hermes）→ 干得稳（Claude Code）

07 多 Agent 架构与 Harness 治理

本章回答：**什么时候该拆多个 Agent、怎么拆、拆完怎么治理**。核心原则一句话：**Agent 负责局部智能，Harness 负责全局控制**。

7.1 主 Agent 与子 Agent 的本质

最重要的认知

子 Agent **不是**主 Agent 的“手”，而是有独立大脑的**协作者**。它具备「**三个独立**」：
独立上下文窗口 · 独立系统提示词 · 独立工具权限。

7.2 通信链路四步（递进）

任务分解 → 上下文传递 → 子 Agent 独立执行 → 结果回传与汇总

① 任务分解

三种策略：按功能域 / 按执行步骤 / 按专业能力。

分解原则

强耦合的步骤留在同一个 Agent；只有**可独立、可并行、上下文互扰**的部分才拆出去。

② 上下文传递（发“任务包”）

任务包字段	说明
任务描述	要做什么，验收标准是什么
必要上下文	完成这个子任务真正需要的信息
工具权限	允许用哪些工具（最小权限）
输出 schema	结果必须是可解析的结构
约束	不能做什么、边界在哪
trace_id	全链路追踪标识

关键在“度”

传太少 → 子 Agent 不理解任务；传太多 → 浪费 Token 还引入噪声。**够用即止**是唯一标准。

③ 子 Agent 独立执行

自主规划、多步推理、中途调整策略 —— 这是它与 Tool 的**根本区别**。主 Agent 不干预过程，只控制**目标、边界、最终验收**。

④ 结果回传与汇总

子 Agent 返回 执行结果 + 执行摘要 + 证据来源 + 置信度 + 失败原因	主 Agent 处理 结果冲突裁决 / 格式统一 / 质量过滤 / 证据校验
--	--

7.3 四种编排模式（并列）

模式	结构	注意点
顺序管道	子 Agent 依次执行，前一个输出是后一个输入	像流水线，适合有明确依赖的流程
Map-Reduce	并行派发 + 全部完成后汇总	最怕 “Reduce 写得太弱” ，汇总没做好等于白干
层级嵌套	子 Agent 再拆孙任务	层数越多成本指数上升， 必须限制深度
路由分发	主 Agent 判类型派给专家 Agent，自己当调度员	主 Agent 只做判断，不干活

7.4 Tool vs 多 Agent（对比）

维度	Tool	Multi-Agent
执行能力	单次调用返回结果	独立规划、多步推理
上下文	结果回到主 Agent 的窗口	子 Agent 独立上下文
自主性	被动执行	主动规划、调整策略
并行性	由外层代码决定	天然适合并行拆分
容错	需主 Agent 额外处理	可按子任务重试 / 降级
协调开销	低	高（通信、汇总、冲突处理）

Agent-as-Tool（中间态）

机制：主 Agent 通过 Function Calling 调一个“工具”，但这个工具内部其实是一个完整的 Agent。

好处	局限
兼顾 Tool 的简单性与 Agent 的自主性	主 Agent 控制力弱，无法中途纠正

前提条件

必须框好四件事：**输入边界 / 工具权限 / 输出 schema / 超时预算**。Claude Code 的 Task 工具就是典型落地。

选型判断（对比）

Tool 够用

步骤固定 · 子任务强依赖 · 延迟敏感 · 需强一致

需要多 Agent

需独立规划 · 上下文互扰 · 需并行加速 · 需专业分工 · 需隔离风险

多 Agent 的代价（必须主动说）

代价	说明
成本与延迟	每个子 Agent 都是独立模型调用，API 量可能 ×3
协调复杂度	任务分解质量 + 上下文粒度 + 冲突处理
调试困难	跨 Agent 日志追踪，难度高一个量级
一致性风险	结论冲突不能简单投票，要做「证据优先级」裁决

真实框架通信（对比）

框架	机制	适合
CrewAI	任务链 / Manager 分配	偏流程编排，流程明确
AutoGen	Agent 间多轮对话，偏协商	需要讨论达成共识，须限最大轮次
Claude Code	主 Agent 通过 Task 调子 Agent	Agent-as-Tool 的工程落地

7.5 Multi-Agent Harness 七层治理 架构岗必答

核心原则与类比

Agent 负责局部智能，Harness 负责全局控制。

类比记忆：Prompt 是台词，Agent 是演员，Tool 是道具，Harness 是导演 + 调度台 + 安全规章 + 预算中心。

层	名称	要点
1	架构编排	Planner 输出声明式计划，Orchestrator 统一裁决；五类决策权：任务生命周期 / 计划裁决 / Agent 路由 / 失败处理 / 硬终止。 硬闸（代码级强制）： <code>max_steps</code>、<code>max_tokens</code>、<code>max_duration</code>、<code>max_tool_calls</code>、<code>max_retries</code> 口诀：别让 Agent 开车，让 Agent 当导航。
2	工具治理	所有工具调用都过 Tool Registry。登记项：名称 / 描述 / 输入 Schema / 允许的 Agent / 超时限速 / 风险等级 / 是否人工确认 / 输出结构 / 审计策略。 原则：给 Agent 一个工具，就是给它一把权限钥匙；哪怕只有 3 个工具也先立规矩。
3	状态与记忆	分清 State（当前任务运行数据）与 Memory（跨任务复用经验）。常见四个坑：记得太少 / 记得太多 / 不分层 / 不遗忘。记忆要有保留分数与修剪机制。
4	评估体系	四层：Component Trajectory (Multi-Agent 重点) Task Completion End-to-End。 LLM-as-Judge 不是万能药，事实正确性优先用确定性检查。 Eval 必须进 CI：Prompts 即代码，Trace 即日志，Eval 即测试。
5	成本控制	Model Routing（简单用小模型、复杂用强模型）；Context Compression（保留最近原文 + 早历史结构化摘要）；预算分级降级。 关键指标：单任务 Token、单 Agent 占比、工具结果占比、重试占比、熔断次数。
6	MCP 接入	MCP 提供能力，Harness 提供治理；标准化 ≠ 安全。 五条最佳实践：Server 不直连 Agent（先过 Harness 过滤） / 工具白名单按 Agent 授权 / 每个 Server 独立配额 / 高风险动作走 HITL / 全链路 Trace。
7	可观测与落地路线	记录：用户输入 / 计划 / 每步输入输出 / 工具参数与结果 / 记忆 / 路由 / Token / 失败与重试 / 降级熔断 / 最终评估。 三阶段落地：Phase1 MVP → Phase2 Hardening → Phase3 Scale（提醒：别一开始就进 Phase 3）。

预算分级降级（第 5 层细节）

绿区正常 → 黄区压缩 → 红区切小模型 → 熔断区强制收束。

08 可观测性与成本优化

本章解决两个线上最痛的问题：**为什么 Demo 能跑通、生产却不稳**，以及**账单为什么爆炸**。

8.1 为什么生产里还是不稳

- 根因不是某一个 Prompt，而是长链路执行过程不可见。
- Agent 的错误会沿执行链路传播：第一步偏 → 全偏。
- Demo 问的是「能不能跑通一次」；生产问的是「能不能稳定跑很多次」。

8.2 可观测性是什么

定义

对**目标 / 计划 / 上下文 / 工具 / 状态 / 成本 / 风险 / 结果质量**的全链路记录、度量、回放、评估。
本质：在 Harness 里给 Agent 装眼睛。
目的不是事后甩锅，而是让每次执行都变成**可分析、可复现、可改进**的工程数据。

	普通日志	Agent 可观测
记录什么	“发生了什么”（接口、参数、状态码、耗时、异常）	还要解释“它为什么这么做”
典型场景	HTTP 全是 200，但任务已经偏了 —— 这就是普通日志看不见、必须上 Agent 可观测的原因	

观测七类对象（并列）

- ① 目标（有无被改写）
② 计划
③ 上下文（来源、是否过期污染）

- ④ 工具（调了什么、参数、返回、耗时）
⑤ 状态变化

- ⑥ 成本
⑦ 评估

8.3 Agent Trace 两层设计（核心技能）

任务级 Trace

字段	含义
trace_id	全链路唯一标识
user_goal / normalized_goal	用户原始目标 / 归一化后的目标（用于检测目标漂移）
agent_version / model_version / policy_version	三个版本号，保证可复现
start_time / end_time / final_status	起止时间与最终状态
total_tokens / total_cost / final_eval	总成本与最终评估

步骤级 Step

字段	含义
step_id / step_type	步骤标识与类型（规划 / 工具 / 观察 / 总结 / 评估）
current_goal / context_refs	当前子目标 / 引用了哪些上下文
model_input_summary / model_output_summary	模型输入输出摘要
tool_name / tool_args / tool_result_summary	工具调用三要素
state_before / state_after	状态变化前后
latency_ms / tokens / risk_level / eval_result	性能、成本、风险等级、本步评估

三个关键字段（面试常问）

context_refs（用了什么上下文，查污染）、state_before/after（状态怎么变的，查一致性）、eval_result（每步质量，查退化点）。

8.4 长链路漂移检测

五信号

相关度下降 · 子任务超预算 · 重复步骤增多 · 目标表述变化 · 关键证据未被引用

三机制

目标锚点 · 检查点评估 · 漂移告警

节奏（工程细节，加分）

不必每步自省，每 3-5 步或异常触发时做一次轻量检测 —— 兼顾成本与灵敏度。

8.5 工具调用故障五类归因（并列）

故障类型	修什么
工具选择错误	修工具描述、路由策略、候选集
参数生成错误	修 JSON Schema、参数校验、错误反馈、示例
权限边界错误	修 Tool Registry、权限策略、高风险审批
工具服务异常	重试、熔断、降级、告警
结果理解错误	记录返回摘要、证据引用、输出校验

8.6 成本监控与优化

成本监控七类指标（并列）

① 总 Token 与单步 Token（总看贵不贵，单步看贵在哪） ② 上下文长度 ③ 工具重试次数 ④ 重复规划次数	⑤ 无效步骤占比 ⑥ 模型路由成本 ⑦ 端到端延迟
--	---------------------------------

成本优化六招（由易到难）

序	手段	效果
①	工具选择优化	只给真正需要的工具，动态加载工具子集
②	模式选择	简单任务用 Workflow 代替 Agent（ 省约 4 倍 Token ）
③	上下文压缩	摘要压缩历史，中间结果只留关键
④	模型路由	简单子任务用小模型
⑤	缓存	工具调用结果缓存 + Prompt 缓存
⑥	预算分层	简单低预算 / 复杂中预算 / 高价值才高预算

混合路由优化（深度加分）

为什么需要 ① 任务难度呈长尾分布，约 80% 是简单任务 ；② 质量提升与成本投入非线性，简单任务小模型几乎不差；③ 一刀切用大模型 = 浪费模型溢价。	三种路由策略（递进） 规则路由： 零成本零延迟、完全可控；但规则维护爆炸、处理不了“伪简单”。 模型路由： 自适应、覆盖长尾；但需训练数据、本身会误判。 混合路由（工程最优）： 规则拦确定简单 + 模型判模糊中间 + 大模型兜底。
--	--

路由模型怎么训练

- **数据来源**：历史调用日志置信度 / 人工标注 / 主动探索（5% 双跑对比）。
- **分类器 vs 评分器**：分类器预测“模型档位”；**评分器预测“难度分”更推荐**——新增模型只需调阈值。
- **冷启动**：先全走大模型并记录置信度 → 训初版 → 只放行高置信请求 → 渐进降阈值。
- **原则**：冷启动宁可多花钱，**质量是底线**。

级联降级（对比）

要点	说明
流程	小模型先试 → 判置信度 → 够高直接用，不够升级大模型
成本算账	一刀切 100 级联 50 理想路由 40
三个坑	双倍延迟 / 阈值难选 / 错误传播
适用	成本极敏感、能容忍延迟；否则混合路由更优

8.7 失败样本沉淀四类资产（并列）

资产	做法
失败归因	定位到目标 / 上下文 / 工具 / 权限 / 状态 / 成本 / 评估
评测样本	变成固定测试用例，改动就跑
Harness 规则	结构性错误补到规则里（如高风险必审批）
监控指标	把早期信号变成告警

8.8 生产级可观测系统六层架构（递进）

层	名称	职责
①	Trace 采集层	在 Harness 关键节点打点，而非散落在 Prompt 里
②	事件存储层	结构化存储，原文脱敏
③	指标计算层	成功率 / 平均步骤 / 平均 Token / 工具失败率 / 漂移率 / 接管率
④	可视化与告警层	研发看回放、业务看成功率与成本、运维看延迟失败率
⑤	回放与评测层	一键回放，固定输入判断新版本是否修好旧问题
⑥	改进闭环层	沉淀到 Prompt / 工具描述 / 权限策略 / 上下文策略 / 评测集 / Harness 规则

09 生产级系统设计与面试答法

本章是终面压轴题的标准答案库：**怎么判断一个 Agent 达到生产级、五层架构怎么搭、评估怎么做、从 0 到 1 怎么走、答题怎么组织。**

9.1 生产级的标准：不是“成功过一次”

第一句话就要说对

Agent 的结果不能由 Agent 自己宣布，必须由外部环境验证。“模型说做完了”不等于“任务真的完成了”。

五道门槛	具体看什么
任务效果	是否真的完成业务目标（不是技术动作完成）
稳定性	模糊输入、工具失败、长链路下结果是否可接受
安全性	能看到什么数据、调什么工具、哪些动作必须人批准
可运营性	延迟 / Token / 成本 / 并发 / 人工接管比例能否长期承受
可改进性	失败能否定位、复现，并进入下一轮修复

9.2 生产级架构五层（递进）

层	名称	职责
①	业务入口与风险分级	身份校验 / 租户隔离 / 输入安全检查 / 风险分级
②	Agent Harness	装配上下文、选模型、暴露工具、维护状态、重试、预算、停止条件、人工确认
③	模型 / 上下文 / 状态 / 记忆	四者不能混在一起
④	工具 / 业务系统 / 执行环境	参数校验、最小权限、超时重试幂等补偿、脱敏、审批、审计
⑤	验证、观测与人工接管	独立验证器 + 完整 Trace + HITL 三要素

9.3 Harness 工程六个转换（递进，非常好用）

从	到
模糊目标	可验收任务
隐性知识	可发现上下文
口头提醒	可执行约束（权限给 IAM、Schema、幂等键、Lint、CI、审批流）
最终回答	可验证结果（能用确定性程序就不用模型反思）
偶发失败	可复现样本（Trace + 隔离环境重放）
一次修复	长期能力（沉淀进规则 / 工具 / 测试 / 评估集）

9.4 生产架构的三个“必须”

必须	说明
确定性与自治并存	外层确定性 Workflow + 内层有限自治 Agent
单 Agent 优先，多 Agent 后置	拆多 Agent 需满足明确条件（见第 7 章）
状态必须外化	Session / Task State / Memory / Artifact / Trace 五者区分
补充一句：权限跟着动作走，不跟着 Agent 名字走。	

9.5 评估四层（递进）

层级	评什么	典型指标
组件评估	单个能力是否正确	工具选择率、参数准确率、检索召回率
轨迹评估	过程是否合理	重复调用、无效步骤、越权尝试、恢复路径
结果评估	外部状态是否真的变了	数据库状态、文件产物、测试结果、业务状态
系统评估	整体能不能长期跑	成功率、 正确失败率 、延迟、成本、人工接管率

最容易被忽略但最专业的指标

正确失败率：信息不足时主动问、权限不够时拒绝、故障时转人工。它衡量的是 Agent “知道自己不知道” 的能力——比成功率更能反映生产成熟度。

Trace 要支持三件事：定位层级 / 重放失败 / 进回归评估。

9.6 组织三层协作

团队	负责	组成 / 产出
场景团队	对业务结果负责	产品 + 领域专家 + 应用工程师
Agent 平台团队	对通用能力负责	网关 / Runtime / Session / Sandbox / Tool Registry / 权限 / Trace / 评估
安全、合规与运营	对边界和持续运行负责	安全策略、合规审查、线上运营

一句话点睛

交接物是版本化资产，不是会议结论。

9.7 人才与能力要求

三类岗位

Agent 应用工程师 · Agent 平台工程师 · Agent 产品与领域专家

六项共同能力

Spec · Context · Tool · Eval · Systems · Domain

最值钱的一句话

稀缺的不是“最会写 Prompt 的人”，而是能把组织经验变成可读、可执行、可验证资产的人。

9.8 企业从 0 到 1 五阶段（递进）

阶段	主题	关键动作
一	先证明任务值得做	判断标准：非结构化信息多、规则难覆盖、可验证、失败成本可控
二	做受控 MVP	单 Agent 或 Workflow + 局部 Agent；只读为主；最小 Trace + 最小评估集
三	补齐 Harness	按高频失败逐个工程化
四	逐级提高自治	建议 → 查询自动 → 低风险自动 + 高风险审批 → 有限自治
五	平台化	多场景出现之后再做，不要提前

9.9 系统设计答题框架（背下来直接用）

目标 → 架构 → Harness → 治理 → 组织

五步走，每一步都能展开成一个子话题，全程不跑题

步骤	说什么
① 目标	业务目标是什么、成功标准是什么（可验收）、失败成本多大、风险等级
② 架构	五层架构：入口风险分级 → Harness → 模型/上下文/状态/记忆 → 工具与执行环境 → 验证观测与接管
③ Harness	六个转换 + 六层组件 + Loop 的五个变量（上下文/状态/预算/工具/终止）
④ 治理	权限最小化、HITL 审批、Trace 可观测、Eval 进 CI、成本预算分级、多 Agent 七层治理
⑤ 组织	场景团队 + 平台团队 + 安全合规；五阶段落地路线；版本化资产沉淀

面试心法（三句话）

- ① 别只背概念，要**把知识点串起来讲**（比如安全 → 权限 → Harness → 可观测是一条线）；
- ② 答题从「**工作模式选择、工具管理、记忆设计、安全防护、成本控制**」五维度展开，保证不漏；
- ③ 底层不变的东西只有四件：**LLM 怎么调工具、工具怎么标准化接入、Agent 之间怎么协作、怎么保证安全可靠**。

10 附录：考前速查

10.1 高频面试题速查表（14 题 • 标准答案）

问题	标准答案要点
LLM 与 Agent 的区别？	LLM 只生成文本（无状态函数）；Agent 在 LLM 之上加了 规划 / 工具 / 记忆 / 循环 ，能改变外部状态
Agent 与 Workflow 怎么选？	看控制权在 LLM 还是代码；生产常用 混合架构 ：外层确定性 Workflow + 内层有限自治 Agent
ReAct 是什么？	Thought → Action → Observation 的循环；本质是 Prompt 范式，循环对象是一段文本
Function Calling 底层怎么工作？	模型输出结构化 JSON 工具调用参数（工具名 + 参数）， 由外部代码执行 ，结果以 role:tool 回传
MCP 解决什么问题？	工具接入标准化 + 动态发现，把 $N \times M$ 变成 $N + M$ ；三个角色：Host / Client / Server
A2A 是什么？	Agent 间横向协作；核心概念 Agent Card + Task + Artifact；与 MCP 互补
Skills 与 Prompt 的区别？	Prompt 是单次指令；Skill 是跨任务复用的能力模块（九大要素）；Skill 是软约束，权限必须工程层强制
记忆系统怎么设计？	上下文窗口（快、有限）+ 向量 DB（语义）+ 关系 DB（精确）+ KV（快状态）；State 与 Memory 分开；混合注入
安全怎么保障？	最小权限原则 + Human in the Loop + 数据/指令分离；Prompt Injection 四手段：分离、过滤、模板隔离、上下文标记
Token 成本怎么优化？	六招：工具选择 / 模式选择 / 上下文压缩 / 模型路由 / 缓存 / 预算分层；核心是混合路由 + 级联降级
生产五大坑是什么？	死循环 / 幻觉工具调用 / 上下文污染 / Token 爆炸 / Prompt Injection
怎么设计一个生产级 Agent？	按五维度展开：工具管理 + 记忆状态 + 可靠性 + 可观测 + 安全；再套「目标→架构→Harness→治理→组织」框架
上下文漂移怎么解？	四步：任务分解 + 子目标检查点 → 上下文压缩 → 定期 Re-Planning → Context Reset（终极解药，状态先外化）
工具调用失败怎么排查？	五步顺序： 工具是否存在 → 权限是否允许 → 参数是否通过 Schema → 工具服务是否成功 → 模型是否正确理解返回

10.2 记忆口诀汇总（考前 30 分钟只看这一页）

总纲

Agent = Model + Harness | 控制的粒度演进：**Prompt** $\subset$ **Context** $\subset$ **Harness**（Loop 是 Harness 的心脏，Loop 降级为 Graph 的节点）

主题	口诀
四层协议	Skills 决定「怎么想」→ MCP 决定「用什么」→ FC 决定「怎么调」；A2A 负责横向连接
FC 本质	模型只说「调哪个、传什么」，执行在应用代码
MCP 价值	$N \times M \rightarrow N + M$ ；三角色 Host / Client / Server；三资源 Tools / Resources / Prompts
状态与记忆	State 管一致性（短命），Memory 管相关性（长命）
幻觉四层防线	Prompt 约束 → 工具约束 → 证据约束 → 输出校验
漂移终极解	重启胜过修补，状态沉到文件里
Harness 三步	看得准（输入）→ 做得对（动作）→ 错了能兜底（校验）
多 Agent 分工	Agent 负责局部智能，Harness 负责全局控制
编排口诀	别让 Agent 开车，让 Agent 当导航
工具授权	给 Agent 一个工具，就是给它一把权限钥匙
图 vs 循环	Loop 没死，它被降到节点内部了
评估即工程	Prompts 即代码，Trace 即日志，Eval 即测试
答题框架	目标 → 架构 → Harness → 治理 → 组织

10.3 关键数字速记（面试报数字最显专业）

数字	对应事实
4–8x	Agent 相比 Workflow 的 Token 消耗倍数 (Workflow $\approx 1x$)
约 20%	Plan-and-Execute 相对 ReAct 的 Token 消耗
约 4 倍	简单任务用 Workflow 代替 Agent 可省的 Token
80%	任务难度长尾分布中简单任务的占比 (混合路由的依据)
100 / 50 / 40	成本算账: 一刀切 / 级联降级 / 理想路由
15 步 (15–50)	最大步数限制的常见阈值
连续 3 次	相同工具 + 相同参数的重复动作检测阈值
128K tokens	上下文窗口的常见容量量级
约 100 行	AGENTS.md 建议长度 (渐进式披露的目录页)
每 3–5 步	长链路漂移检测的节奏
5%	路由模型训练时主动探索的双跑对比比例
23 层检查	Claude Code 的顺序权限检查层数
200K 窗口	Claude Code 的上下文窗口 + 三层压缩
25+ 平台	OpenClaw 一个网关打通的消息平台数
N + M	MCP 把工具集成从 $N \times M$ 降到的复杂度

10.4 术语表

术语	一句话解释
ReAct	Thought-Action-Observation 循环的 Prompt 范式
Function Calling	模型输出结构化工具调用指令的机制
MCP	工具接入标准化协议 (Host / Client / Server)
A2A	Agent 间横向协作协议 (Agent Card / Task / Artifact)
Human in the Loop	高风险操作暂停等待人工审批
Context Reset	丢弃旧上下文换干净窗口，唯一前提是状态已外化
Harness	除模型外的一切工程外壳，Agent – Model
Auto-Compact	上下文超阈值时自动压缩
Trace	全链路结构化执行记录，任务级 + 步骤级两层
Eval	可回归的评测集，必须进 CI
Tool Registry	所有工具的统一注册与治理中心
Agent-as-Tool	把完整 Agent 包装成一个工具供主 Agent 调用
Golden Principles	对抗技术债的固定原则，由后台 Agent 持续执行
正确失败率	信息不足主动问、权限不够拒绝、故障转人工的比例

最后一句：学习顺序的收敛点

如果时间只够记一句话，就记这个：
LLM 只能生成；Function Calling 让它能调工具；MCP 把工具接入标准化；记忆与规划让它能坚持多步；Harness 让它能稳定地跑很多次。
所有细节，都挂在这条线上。